A Non-Compact Formulation and Complexity Framework for the Tool Switching Problem

Shankar Narayanan

Internship Report

Advisors: Prof. Rafael Colares and Prof. Annegret Wagler

March 30, 2026

Abstract

The management of tool configurations in Flexible Manufacturing Systems (FMS) is a well-studied NP-hard combinatorial optimization challenge. This report provides a mathematical analysis of the sequential Tool Switching Problem (SSP) by evaluating its non-compact formulation. We systematically establish the structural equivalences of the SSP and its nested subproblems to classical optimization frameworks, including the Generalized Traveling Salesman Problem (GTSP), the k-Server Paging Problem, totally unimodular Network Flows, and the Set Union Knapsack Problem (SUKP). By mapping the problem into these established domains, we construct an exact Branch-and-Price framework, utilizing Lexicographical Minimization over the Job Grouping Problem (JGP) for bound initialization, and identify precise pathways for Machine Learning acceleration within the NP-hard pricing oracle.

1 Introduction

Flexible Manufacturing Systems (FMS) enable rapid adaptation to market changes by utilizing numerically controlled machines equipped with automated tool magazines. Because a single machine may be required to process a diverse batch of parts, and because the total number of distinct tools required often exceeds the capacity of the machine's magazine, tool switches are unavoidable (Tang and Denardo, 1988). A tool switch incurs a time penalty that delays production. Consequently, optimal tool management is a primary objective in operations research.

The literature broadly divides tool management into two distinct problems based on operational constraints:

1. **The Job Grouping Problem (JGP):** Occurs when a machine must be entirely stopped to change tools, allowing multiple tools to be swapped simultaneously. The objective is to partition the jobs into a minimum number of feasible groups, effectively treating the issue as a Set Covering problem (Crama and Oerlemans, 1994).

2. **The Tool Switching Problem (SSP):** Models environments where jobs are processed sequentially and tool switches occur individually. The objective is to simultaneously determine the optimal job processing sequence and the tool loading policy to minimize the total number of individual tool switches (Catanzaro et al., 2015).

This report focuses on the uniform Job Sequencing and Tool Switching Problem (SSP). In this problem, a set of jobs must be processed on a single machine equipped with a tool magazine of limited capacity. Each job requires a specific subset of tools, and all required tools must be present in the magazine before the job can be processed. The objective is to determine a processing sequence for the jobs and an associated tool loading policy that jointly minimize the total number of tool switches.

Because the SSP is strongly NP-hard (Crama and Oerlemans, 1994), modern solution methodologies rely on decomposition. As formally proven by Tang and Denardo (1988), the SSP naturally decomposes into two nested subproblems: the **Sequencing Problem (SP)** (determining the optimal job order) and the **Tooling Problem (TP)** (determining the optimal tool loading and eviction policy for a fixed sequence).

To develop a state-of-the-art exact solver, it is imperative to move beyond isolated SSP heuristics. The primary objective of this report is to analyze the non-compact SSP formulation by proving its mathematical equivalence to other classical Combinatorial Optimization (CO) domains. Specifically, we will demonstrate how the TP maps to k-Server memory paging and Max-Flow Min-Cost networks, how the SP maps to the GTSP, and how the column-generation pricing oracle structurally equates to the Set Union Knapsack Problem. By establishing these equivalences, we adapt state-of-the-art solution ideas into a unified Branch-and-Price (B&P) framework and identify targeted interventions for Machine Learning.

2 The Colares-Wagler Non-Compact Formulation

The computational complexity of the SSP is fundamentally rooted in its state space. Traditional compact models sequence the jobs directly and track tool insertions step-by-step.

However, these models suffer from symmetry caused by 0-blocks. As formally defined by Crama and Oerlemans (1994), a 0-block is a maximal subset of consecutive zeros in a row of the tool-job incidence matrix—representing a time interval before and after which a tool is needed, but during which it is not required.

The act of retaining an unneeded tool in the magazine across this gap to avoid redundant unloading and reloading is mathematically referred to as "flipping" the 0-block to 1s. This structure forces compact solvers to evaluate highly symmetric, redundant permutations to decide which 0-blocks to flip, severely limiting computational performance (Catanzaro et al., 2015; Laporte et al., 2004).

To establish an intuitive foundation before introducing the formal mathematics, consider the small instance illustrated in Figure 1.

0-block (Gap of zeros)

	j_1	j_2	j_3	j_4	j_5
t_1	1	0	0	1	1
t_2	1	1	0	0	1
t_3	1	1	0	0	0
t_4	0	1	1	0	0
t_5	0	0	1	1	0
t_6	0	0	1	1	0
t_7	0	0	0	0	1

Figure 1: An incidence matrix for a 5-job, 7-tool instance. The machine magazine has a capacity $b = 4$. Assuming the sequence $j_1 \to j_2 \to j_3 \to j_4 \to j_5$, tool t_1 is required by jobs j_1, j_4, and j_5, but not by j_2 and j_3. This gap of non-requirement represents a 0-block for t_1. Because $b = 4$ is large enough to accommodate the transition tool requirements, the optimal policy evaluates whether to "flip" this 0-block by retaining t_1 through jobs j_2 and j_3 to avoid redundant setups.

2.1 Illustrative Example: Magazine State Evolution

To illustrate the tool switching mechanics and the concrete impact of flipping a 0-block, consider the 5-job, 7-tool instance previously introduced in Figure 1. The machine has a magazine capacity of $b = 4$, and the jobs are processed in the sequence $j_1 \to j_2 \to j_3 \to j_4 \to j_5$. The specific tool requirements for each job are as follows:

- $T_1 = \{t_1, t_2, t_3\}$
- $T_2 = \{t_2, t_3, t_4\}$

- $T_3 = \{t_4, t_5, t_6\}$
- $T_4 = \{t_1, t_5, t_6\}$
- $T_5 = \{t_1, t_2, t_7\}$

Under the optimal Keep Tool Needed Soonest (KTNS) policy (Tang and Denardo, 1988), the solver looks ahead to determine which 0-blocks to flip to minimize future swaps. The evolution of the optimal magazine configurations across this sequence is depicted in Figure 2.

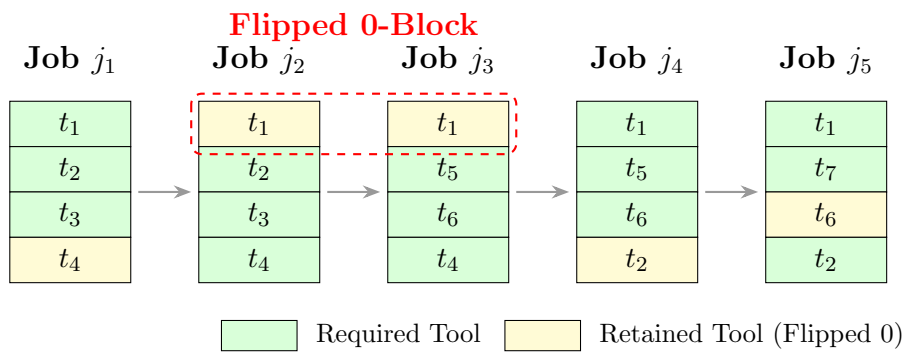

Figure 2: State-space timeline of the tool magazine. Here, tool t_1 is safely retained across its 0-block (strictly during jobs j_2 and j_3) because the magazine capacity is large enough to accommodate the transition tools. This flips the 0-block to 1s in the state space, avoiding redundant unloading and reloading operations.

To bypass the sequence-based symmetry inherent in determining which 0-blocks to flip step-by-step, we utilize the non-compact formulation proposed by Colares and Wagler (2026), which projects the problem entirely onto a complete directed graph of valid configurations.

2.2 Definitions, Variables, and Maximal Configurations

Let $J = \{1, 2, \ldots, n\}$ be the set of jobs and $T = \{1, 2, \ldots, m\}$ be the set of all available tools. Each job $j \in J$ requires a specific subset of tools $T_j \subseteq T$. The machine's tool magazine has a capacity b.

A fundamental property of this formulation is the assumption of maximal configurations. Let $\mathcal{C} = \{C \subseteq T \mid |C| \leq b\}$ be the collection of all valid tool configurations. Without loss of generality, we establish that the tool repository is always full, i.e., $|C| = b$ for all considered configurations (Colares and Wagler, 2026).

Proposition 2.1 (Configuration Maximization)**.** *Any configuration C where $|C| < b$ is strictly dominated by a maximal configuration $C' = C \cup T_{dummy}$ where $|C'| = b$.*

Proof. Let $d(C_1, C_2)$ be the number of tool switches required to transition from configuration C_1 to C_2. The cost formula evaluates to $d(C_1, C_2) = b - |C_1 \cap C_2|$. If $|C_1| < b$, the empty slots can be filled with arbitrary tools from $T \setminus C_1$ to form C_1', yielding $|C_1'| = b$. Because $C_1 \subseteq C_1'$, it mathematically follows that $|C_1 \cap C_2| \leq |C_1' \cap C_2|$. Therefore, $b - |C_1' \cap C_2| \leq b - |C_1 \cap C_2|$. Padding an under-capacity configuration incurs a mathematical penalty of zero while strictly increasing the probability that the added tools may satisfy subsequent job requirements, thereby avoiding future switches (Tang and Denardo, 1988). Thus, we can safely assume $|C| = b$ for all $C \in \mathcal{C}$. □

We consider the complete graph K_p, where each vertex $v \in V(K_p)$ corresponds to a distinct maximal configuration $C \in \mathcal{C}$. The edge weight $d(u, v)$ represents the number of tool switches required to transition from configuration C_u to C_v:

$$d(u, v) = b - |C_u \cap C_v| \tag{1}$$

Let $H_j \subseteq V(K_p)$ denote the subset (cluster) of configurations capable of processing job $j \in J$. That is, $H_j = \{C_i \in \mathcal{C} \mid T_j \subseteq C_i\}$. The formulation utilizes two binary decision variables: $y(v) \in \{0, 1\}$ indicating whether configuration v is used to process some job, and $x(u, v) \in \{0, 1\}$ indicating whether configuration v is applied right after configuration u in the optimal sequence.

2.3 Mathematical Model and Correctness

The non-compact SSP is formulated over the complete graph K_p as follows:

$$\min \quad \sum_{u \in V(K_p)} \sum_{v \in V(K_p)} d(u, v) x(u, v) \tag{2}$$

$$\text{s.t.} \quad \sum_{v \in H_j} y(v) \geq 1 \qquad \forall j \in J \tag{3}$$

$$\sum_{u \in V(K_p)} x(u, v) = y(v) \qquad \forall v \in V(K_p) \tag{4}$$

$$\sum_{u \in V(K_p)} x(v, u) = y(v) \qquad \forall v \in V(K_p) \tag{5}$$

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \leq \sum_{v \in H(S)} y(v) - 1 \qquad \forall S \subseteq J, 2 \leq |S| \leq |J| - 1 \tag{6}$$

$$x(u, v) \in \{0, 1\} \qquad \forall u, v \in V(K_p) \tag{7}$$

$$y(v) \in \{0, 1\} \qquad \forall v \in V(K_p) \tag{8}$$

where $H(S) = \bigcup_{j \in S} H_j$.

To establish the correctness of this formulation, we must prove that the constraints mathematically enforce a valid, continuous sequence of configurations that covers all jobs, and that the objective evaluates the optimal tool switching cost without loss of generality.

Lemma 2.1 (Flow and Cover Feasibility)**.** *Constraints* (3)*,* (4)*, and* (5) *ensure that the active configurations ($y(v) = 1$) form a collection of node-disjoint directed cycles such that every job $j \in J$ is covered by at least one active configuration.*

Proof. By constraint (3), for every $j \in J$, there exists at least one configuration $v \in H_j$ such that $y(v) \geq 1$. Since $y(v) \in \{0, 1\}$, this forces $y(v) = 1$. Constraints (4) and (5) are flow conservation equations. They dictate that if a configuration v is active ($y(v) = 1$), it must have exactly one incoming edge and one outgoing edge. If $y(v) = 0$, it has no incident edges. In any finite directed graph, a subgraph where every active vertex has an in-degree and out-degree of exactly 1 inherently decomposes into a set of node-disjoint directed cycles. Therefore, the active configurations form a set of closed cycles that collectively cover all job tool requirements. □

Lemma 2.2 (Subtour Elimination)**.** *Constraint* (6) *guarantees that the active configurations form exactly one connected Hamiltonian cycle.*

Proof. Suppose, for the sake of contradiction, that a feasible solution satisfying (3)–(5) forms $k \geq 2$ disjoint directed cycles, $W_1, W_2, \ldots, W_k$. Let $V(W_1)$ be the set of active vertices in cycle W_1. Let $S \subset J$ be the proper subset of jobs covered exclusively by the configurations in W_1. Because $k \geq 2$ and the clusters H_j partition the job requirements, W_1 cannot process all jobs without rendering the other cycles completely redundant. Thus, $2 \leq |S| \leq |J| - 1$.

By definition, all vertices in W_1 belong to $H(S) = \bigcup_{j \in S} H_j$, meaning $V(W_1) \subseteq H(S)$. If we sum the selected edges $x(u, v)$ within W_1, we have:

$$\sum_{u \in V(W_1)} \sum_{v \in V(W_1)} x(u, v) = |V(W_1)| = \sum_{v \in V(W_1)} y(v) \tag{9}$$

Since $V(W_1) \subseteq H(S)$, evaluating the Subtour Elimination Constraint (6) strictly over S yields:

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \geq \sum_{v \in V(W_1)} y(v) \tag{10}$$

However, constraint (6) strictly bounds the sum of the edges from above, demanding $\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v$ $\sum_{v \in H(S)} y(v) - 1$. This constitutes a direct contradiction. Thus, the formulation strictly forbids disjoint sub-cycles, enforcing $k = 1$. □

Theorem 2.3 (Correctness)**.** *The integer linear program defined by* (2)–(8) *correctly identifies an optimal job sequence and tool-loading policy that minimizes the total number of tool switches for the SSP.*

Proof. By Lemma 2.1 and Lemma 2.2, the formulation's constraints define a single, connected cycle over a subset of configuration vertices that collectively satisfy all job requirements. For any two consecutively applied configurations C_u and C_v (where $x(u, v) = 1$), the number of tools that must be inserted into the magazine is exactly the capacity b minus the number of tools retained from the previous configuration, $|C_u \cap C_v|$. Thus, the transition edge cost $d(u, v) = b - |C_u \cap C_v|$ exactly computes the number of physical tool switches required. The objective function (2) accumulates these transition costs along the active edges of the cycle. Therefore, minimizing this objective over the feasible space correctly yields a sequence mapping directly to the minimum number of tool switches. □

3 Graph Transformations and Complexity

To formally analyze the algorithmic pathways for solving the SSP, we must establish its computational relationship with the Generalized Traveling Salesman Problem (GTSP). While the problems share a covering-and-routing structure, their state spaces differ fundamentally. This section details the graph transformations that link the two problems, formalizes the exponential complexity of these mappings, and establishes the complexity hierarchy via structural subsumption.

3.1 Formal Definition of Instances

We formally define an instance of the sequential SSP as $\mathcal{I}_{SSP} = \langle J, T, b, \{T_j\}_{j \in J} \rangle$, where $J = \{1, \ldots, n\}$ is the set of jobs, $T = \{1, \ldots, m\}$ is the set of tools, b is the magazine capacity, and $T_j \subseteq T$ is the specific tool requirement for each job.

Conversely, we define an instance of the GTSP as $\mathcal{I}_{GTSP} = \langle V, E, d, \{V_k\}_{k \in K} \rangle$, where V is the vertex set partitioned into mutually exclusive clusters V_k ($V_i \cap V_j = \emptyset$), E is the set of directed edges, and $d : E \to \mathbb{R}^+$ is the transition cost function. The objective is to find a minimum-cost cycle visiting exactly one vertex from each cluster.

3.2 State-Space Transformation: From SSP to GTSP

In the SSP, a single maximal tool configuration $C \in \mathcal{C}$ can contain the necessary tools to satisfy multiple jobs simultaneously. Therefore, the configuration clusters H_j are inherently

overlapping ($H_i \cap H_j \neq \emptyset$). To map this overlapping state space into a disjoint routing problem, we adapt the Intersecting-to-Nonintersecting (I-N) transformation (Lien et al., 1993).

The Construction: We construct the transformed vertex set V' by duplicating valid SSP configurations into job-specific replicas. These vertices are partitioned into exactly $n = |J|$ clusters. Each cluster V_j corresponds uniquely to a job $j \in J$:

$$V_j = \{v_{C,j} \mid C \in \mathcal{C},\ T_j \subseteq C\} \quad \forall j \in J \tag{11}$$

Because each replica vertex is uniquely indexed by its job j, the transformed clusters are strictly disjoint ($V_i \cap V_j = \emptyset$ for $i \neq j$). The distance function d' between any two replicas in V' is defined by the underlying transition cost of their configurations:

$$d'(v_{C_u,i}, v_{C_w,j}) = b - |C_u \cap C_w| \tag{12}$$

Crucially, intra-cluster edges connecting replicas of the exact same configuration are assigned a cost of zero. This transformation is illustrated in Figure 3.

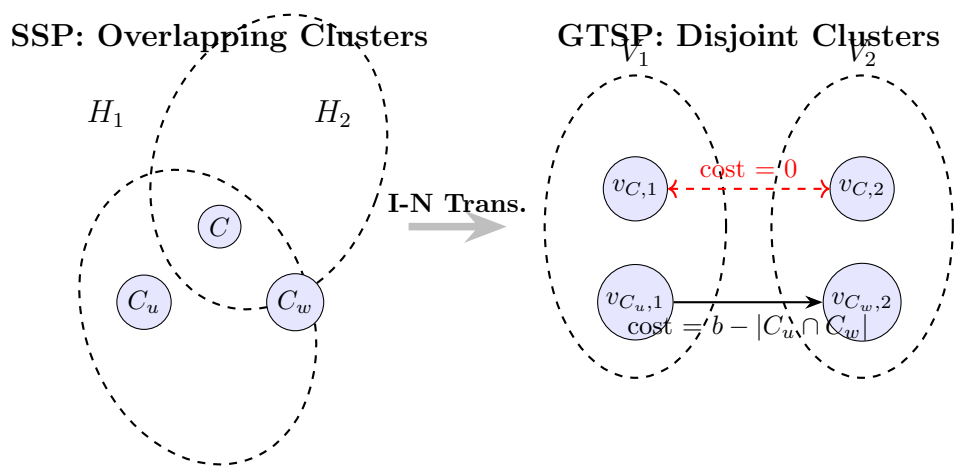

Figure 3: The Intersecting-to-Nonintersecting (I-N) transformation. A configuration C capable of satisfying both Job 1 and Job 2 is duplicated into job-specific replicas $v_{C,1}$ and $v_{C,2}$ in strictly disjoint clusters, connected by a zero-cost edge.

Lemma 3.1 (Non-Redundant Subtours)**.** *If a GTSP tour visits two replicas of the same configuration non-consecutively, it can always be transformed into a non-redundant tour of equal or lesser cost where all replicas of that configuration are visited consecutively.*

Proof. Let $v_{C,i}$ and $v_{C,j}$ be replicas of the same configuration C. Their internal distance is $d'(v_{C,i}, v_{C,j}) = b - |C \cap C| = 0$. Because the distance metric intrinsically satisfies the triangle

inequality (Lemma 2.1), bypassing intermediate nodes to visit $v_{C,j}$ immediately after $v_{C,i}$ via the zero-cost edge will never increase the total tour cost. Repeated shortcutting groups all replicas of C into a continuous zero-cost subtour (Lien et al., 1993). □

Theorem 3.2 (Optimal Solution Correspondence)**.** *A minimum-cost GTSP tour on the transformed graph corresponds to an optimal SSP sequence without loss of optimality.*

Proof. By Lemma 3.1, an optimal GTSP tour can be made non-redundant. In a non-redundant tour, all replicas of a chosen configuration C are visited consecutively, incurring an internal transition cost of 0 between them. This perfectly emulates the Keep Tool Needed Soonest (KTNS) policy, which retains tool configurations across a 0-block of jobs at zero penalty (Tang and Denardo, 1988). Consequently, this non-redundant sequence of disjoint clusters collapses directly into a valid, sequence-preserving tool-loading policy for the SSP with identical cost. □

Complexity of the Transformation: While the I-N transformation establishes a theoretical correspondence, it is critical to quantify its algorithmic complexity. The maximum number of valid configurations is bounded by $\binom{m}{b}$. Replicating these across n jobs yields a transformed state space strictly bounded by $\mathcal{O}(n\binom{m}{b})$. Because this state-space expansion requires exponential time and space, it operates as an algorithmic formulation technique for routing solvers rather than a polynomial-time Karp reduction.

3.3 Theoretical Equivalence and Structural Subsumption

To investigate whether the SSP and GTSP are strictly equivalent complexity classes, one must analyze the reverse mapping: converting a GTSP into the SSP.

The Restricted Reverse Construction: A reverse mapping is theoretically possible under highly restricted conditions. If a GTSP instance possesses strictly integer, symmetric, and Euclidean edge weights, it can be mapped to the SSP by artificially engineering dummy tools.

Lemma 3.3 (Dummy Tool Padding)**.** *For any symmetric GTSP edge weight $w_{uv} \in \mathbb{Z}^+$ bounded by b, there exists a theoretical SSP configuration space where the transition cost matches the edge weight: $b - |C_u \cap C_v| = w_{uv}$.*

Proof. If we assign a unique, disjoint set of w_{uv} dummy tools to configurations C_u and C_v, while filling their remaining $b - w_{uv}$ slots with identical shared tools, the resulting set intersection is exactly $|C_u \cap C_v| = b - w_{uv}$. Applying the transition formula yields $b - (b - w_{uv}) = w_{uv}$, perfectly matching the edge weight. □

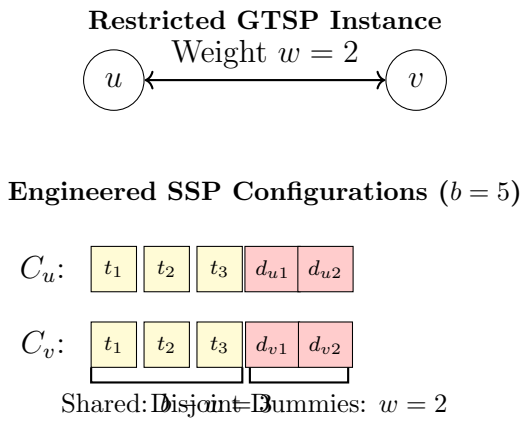

Figure 4: The theoretical restricted reverse construction. By generating disjoint dummy tools for specific configurations, the SSP transition metric $b - |C_u \cap C_v|$ can be engineered to match integer GTSP edge weights. This construction requires exponential time to encode a full graph.

The Metric Barrier: While Lemma 3.3 demonstrates a mapping, a general polynomial reduction from the GTSP to the SSP is mathematically impossible. The barrier lies in the fundamental nature of the transition costs. By Lemma 2.1, the SSP's transition metric $(b - |C_u \cap C_v|)$ is strictly integer-bounded, inherently symmetric, and naturally satisfies the triangle inequality. In stark contrast, a general GTSP allows for arbitrary, real-valued, and highly asymmetric edge weights. It is mathematically impossible to polynomially construct dummy tools whose set intersections can capture arbitrary real-valued, asymmetric metrics.

Because a general reverse reduction is prohibited by the metric barrier, the formal complexity hierarchy between the two problems is established not by artificial graph transformations, but by structural subsumption. The GTSP can be seen as a particular case of the Tool Switching Problem where the clusters H_j are all disjoint (Colares and Wagler, 2026). We formally prove that this disjointness condition occurs naturally without any artificial edge-weight mapping.

Proposition 3.1 (The Disjointness Condition)**.** *If the union of the required tools for any two distinct jobs $i, j \in J$ strictly exceeds the magazine capacity, i.e., $|T_i \cup T_j| > b$, then their corresponding configuration clusters H_i and H_j are strictly disjoint ($H_i \cap H_j = \emptyset$).*

Proof. Suppose, for the sake of contradiction, that $|T_i \cup T_j| > b$ holds, yet the clusters are not disjoint ($H_i \cap H_j \neq \emptyset$). This implies there exists at least one valid tool configuration $C \in \mathcal{C}$ such that $C \in H_i$ and $C \in H_j$. By definition of the clusters, this requires $T_i \subseteq C$ and $T_j \subseteq C$. It mathematically follows that their union must also be a subset of the configuration: $(T_i \cup T_j) \subseteq C$. Consequently, the cardinality must satisfy $|T_i \cup T_j| \leq |C|$. Since every valid configuration is strictly bounded by the magazine capacity ($|C| = b$), we

arrive at $|T_i \cup T_j| \leq b$. This directly contradicts the initial premise that $|T_i \cup T_j| > b$. Therefore, no such configuration C can exist, and the clusters must be strictly disjoint. $\square$

Under the Disjointness Condition, no valid tool configuration can possess the capacity to process more than one job. The overlapping nature of the SSP's state space completely vanishes. The problem structurally collapses into finding the shortest Hamiltonian cycle by visiting exactly one configuration vertex from a collection of mutually exclusive, disjoint clusters—which is the exact formal definition of the GTSP. Because the GTSP is a restricted state of the SSP, the SSP operates as the more computationally generalized framework.

3.4 Algorithmic Implications of the Complexity Space

Because the SSP is structurally more generalized and its forward state-space transformation (the I-N mapping) requires exponential memory, generating the complete configuration graph is only computationally viable for smaller, tightly-constrained instances. This inherent complexity dictates a bifurcated solution methodology (illustrated in Figure 5).

For tractable instances, the complete GTSP graph and its mutually exclusive clusters can be generated in memory. We then apply the Noon-Bean transformation (Noon and Bean, 1993), a graph transformation that modifies the edge weights to compel any optimal routing algorithm to visit exactly one node per cluster. This collapses the generalized problem into a standard Asymmetric Traveling Salesman Problem (ATSP), which can be solved optimally using state-of-the-art routing heuristics such as LKH-3 (Helsgaun, 2015).

However, for massive industrial instances where the combination of m and b triggers out-of-memory failures, exactness requires bypassing the full graph via dynamic node generation in a Branch-and-Price paradigm.

4 Relationship with the Job Grouping Problem (JGP)

To fully contextualize the non-compact formulation and the necessity of dynamic column generation for the SSP, it is critical to establish its relationship with the Job Grouping Problem (JGP). Historically, Column Generation in flexible manufacturing was pioneered to solve the JGP by treating it as a Set Covering problem, where each generated column represents a feasible tool configuration capable of processing a subset of jobs (Crama and Oerlemans, 1994).

While the SSP's Restricted Master Problem (RMP) utilizes these exact same generated configurations as its vertices, the fundamental objectives and transition mechanics of the

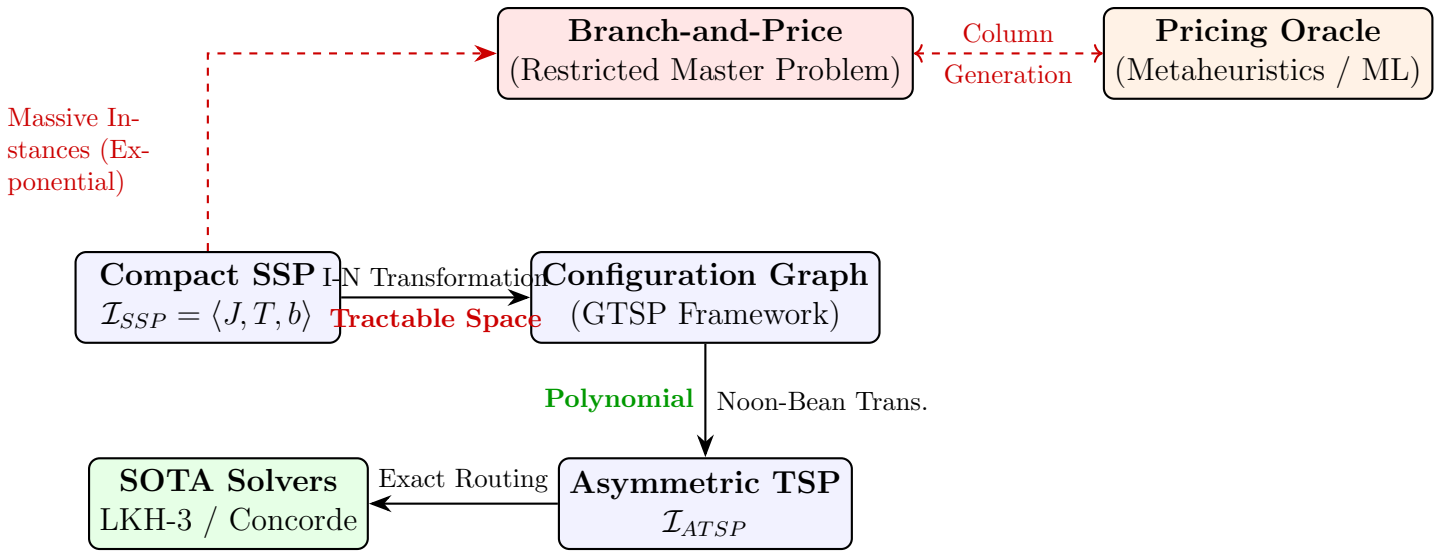

Figure 5: The complexity space of solving the SSP. Tractable instances can be reduced to ATSP and solved via highly optimized routing solvers. Massive instances bypass the exponential bottleneck via a Branch-and-Price framework, where the NP-hard pricing oracle is solved dynamically using metaheuristics or Machine Learning (Shen et al., 2022).

two problems differ significantly, which justifies why a pure JGP solution is insufficient for the SSP.

4.1 Conceptual Distinction and Suboptimality of JGP

The JGP aims to partition a set of jobs into the minimum number of feasible groups such that all jobs within a group can be processed with a single setup of the tool magazine. It assumes the machine is completely halted between groups to allow for simultaneous, batch tool swaps. In contrast, the SSP seeks to determine a continuous processing sequence that minimizes individual tool switches, dynamically retaining unneeded tools across transitions via policies such as the Keep Tool Needed Soonest (KTNS) rule (Tang and Denardo, 1988).

Because the JGP forces jobs into rigid batches, it locks the magazine configuration for the duration of the group. This rigidity can temporarily choke the magazine capacity and force the unnecessary eviction of tools that will be required shortly after, leading to strict suboptimality when evaluated under continuous transitions.

To prove this, consider an instance with a magazine capacity of $b = 3$, containing 6 jobs and 6 tools that form a cyclical requirement ring:

- $T_1 = \{t_1, t_2\}$
- $T_2 = \{t_2, t_3\}$
- $T_3 = \{t_3, t_4\}$

- $T_4 = \{t_4, t_5\}$
- $T_5 = \{t_5, t_6\}$
- $T_6 = \{t_6, t_1\}$

The strict objective of the JGP is to partition the jobs into a minimum number of feasible groups. Because every 3 consecutive jobs require 4 distinct tools (e.g., $T_1 \cup T_2 \cup T_3 = \{t_1, t_2, t_3, t_4\}$), no 3 jobs can fit into a single group. The absolute minimum number of groups is 3 batches of 2 jobs each:

1. **Group 1 (Jobs 1 & 2):** Requires configuration $\{t_1, t_2, t_3\}$.
2. **Group 2 (Jobs 3 & 4):** Requires configuration $\{t_3, t_4, t_5\}$.
3. **Group 3 (Jobs 5 & 6):** Requires configuration $\{t_1, t_5, t_6\}$.

Evaluating the continuous transition cost between these rigid JGP batches:

- **G1 $\rightarrow$ G2:** The magazine transitions from $\{t_1, t_2, t_3\}$ to $\{t_3, t_4, t_5\}$. Tools t_1 and t_2 are dropped to load t_4 and t_5 (2 switches).
- **G2 $\rightarrow$ G3:** The magazine transitions from $\{t_3, t_4, t_5\}$ to $\{t_1, t_5, t_6\}$. Tools t_3 and t_4 are dropped to load t_1 and t_6 (2 switches).

The total cost of the JGP grouping is **4 tool switches**. During the transition from G1 to G2, the rigid batch requirement of Group 2 completely fills the magazine, forcing tool t_1 out, even though it is required for Group 3.

Conversely, if the SSP processes the same sequence of jobs sequentially using the dynamic KTNS policy, the configuration shifts continuously:

- **Job 1 to Job 2:** Magazine remains $\{t_1, t_2, t_3\}$ (0 switches).
- **Job 3:** The magazine drops t_2 (never needed again) and loads t_4. Magazine becomes $\{t_1, t_3, t_4\}$ (1 switch). Tool t_1 is safely retained.
- **Job 4:** Drops t_3, loads t_5. Magazine becomes $\{t_1, t_4, t_5\}$ (1 switch).
- **Job 5:** Drops t_4, loads t_6. Magazine becomes $\{t_1, t_5, t_6\}$ (1 switch).
- **Job 6:** Magazine is already $\{t_1, t_5, t_6\}$. Requirements are met (0 switches).

The total SSP cost is **3 tool switches**. By dynamically shifting the configuration and safely retaining tool t_1 in the corner of the magazine, the SSP bypasses the rigid capacity choke of the JGP, proving that $Cost_{switches}(SSP) \leq Cost_{switches}(JGP)$.

4.2 Approximation Bounds from JGP to SSP

While an optimal JGP solution cannot guarantee an optimal SSP sequence, it provides a strict and reliable theoretical bound. We can mathematically establish that the optimal JGP solution yields an α-approximation for the SSP, where $\alpha = b$ (the magazine capacity).

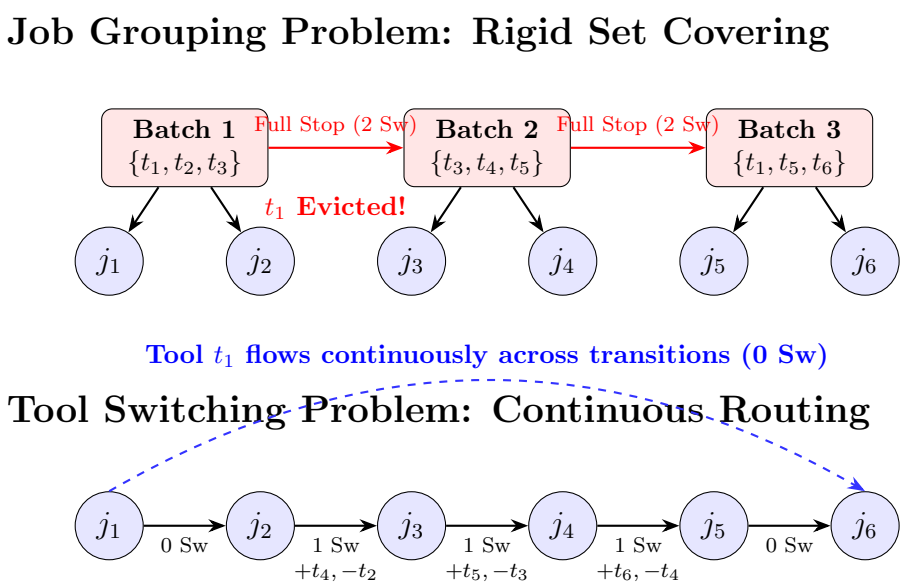

Figure 6: Graph-theoretic contrast between the Job Grouping Problem (JGP) and the Tool Switching Problem (SSP). The JGP acts as a Set Covering problem, mapping jobs to rigid, independent configuration nodes separated by full machine stops. The SSP models a continuous routing path, evaluating dynamic edge costs between individual jobs and allowing specific tools (such as t_1) to flow seamlessly across the sequence.

Let S_{SSP} be the optimal number of tool switches in the SSP. Let g_{JGP} be the optimal (minimum) number of groups in the JGP.

Step 1: Bounding JGP groups using SSP switches. In an optimal SSP schedule, the magazine state changes exactly S_{SSP} times, creating a timeline of $S_{SSP} + 1$ distinct magazine configurations. Because every job is processed entirely within one of these configurations, and every configuration strictly respects the magazine capacity b, these $S_{SSP} + 1$ configurations represent a valid JGP grouping. Since g_{JGP} is the absolute minimum number of feasible groups required, it must mathematically hold that:

$$g_{JGP} \leq S_{SSP} + 1 \implies g_{JGP} - 1 \leq S_{SSP} \tag{13}$$

Step 2: Bounding JGP switch cost using JGP groups. In the absolute worst-case scenario, transitioning between two completely disjoint JGP groups requires a complete machine stop and a total flush of the magazine, costing exactly b tool switches. Because there are g_{JGP} groups, there are exactly $g_{JGP} - 1$ transitions. Therefore, the maximum possible number of tool switches forced by the JGP solution is:

$$Cost_{switches}(JGP) \leq (g_{JGP} - 1) \times b \tag{14}$$

Step 3: The Approximation Bound. Substituting the inequality from Step 1 into Step 2 yields $Cost_{switches}(JGP) \leq S_{SSP} \times b$. Dividing both sides by S_{SSP} establishes the final bound:

$$\frac{Cost_{switches}(JGP)}{S_{SSP}} \leq b \tag{15}$$

This deduction proves that any JGP optimal solution will always provide an α-approximation for the SSP strictly bounded by the size of the tool magazine b. Consequently, the JGP Set Covering formulations serve as a highly effective initial construction phase. Modern solution frameworks can initialize their Branch-and-Price structures using these JGP configurations (columns), subsequently refining them via GTSP routing logic and metaheuristic improvement strategies to close the optimality gap.

4.3 Tightening the Bounds: From Heuristics to Exact Methods

While the α-approximation establishes a theoretical worst-case guarantee, solving the SSP in practice requires aggressively closing the gap between the JGP batching formulation and the continuous SSP routing sequence. In a Branch-and-Price framework, algorithmic efficiency dictates that we establish tight upper bounds (primal bounds) to prune the search tree, and tight lower bounds (dual bounds) to prove optimality.

Heuristic Upper Bounds. Because a valid JGP batching solution can be flattened into a linear job sequence, evaluating that sequence using the KTNS policy inherently provides a valid upper bound for the SSP. However, owing to the rigid batch boundaries of the JGP, this naive upper bound is rarely tight. To overcome this, early research proposed using JGP solutions merely as an initial construction phase, which is subsequently refined by improvement heuristics. Methods such as the "Super Task" heuristic (Privault and Finke, 1995) and global 2-opt local search (Crama and Oerlemans, 1994) were deployed to break the rigid JGP batch boundaries, allowing the algorithm to rearrange jobs and exploit overlapping tool requirements.

However, pure local search is highly susceptible to local optima. To establish the tightest heuristic upper bounds, the current state-of-the-art relies on hybrid population-based metaheuristics, such as Hybrid Genetic Search (HGS) (Mecler et al., 2021) and Adaptive Large Neighborhood Search (ALNS) (Beezão et al., 2017). Alternatively, by mapping the configurations to a GTSP, highly optimized routing heuristics such as LKH-3 (Helsgaun, 2015) can be applied to find near-optimal continuous sequences. Injecting these high-quality sequences as initial primal bounds drastically reduces the size of the exact search tree.

Strengthening Lower Bounds. Historically, establishing tight lower bounds for the SSP has been a severe computational bottleneck. The original Integer Linear Programming

(ILP) formulations, such as those proposed by Tang and Denardo (1988), suffered from notoriously weak linear relaxations. The "zero-bound" LP relaxation of early compact models forced branch-and-bound algorithms into nearly complete enumeration.

Subsequent literature sought to overcome this by introducing valid inequalities. Laporte et al. (2004) significantly tightened the lower bound by adding subtour elimination constraints, alongside look-ahead capacity and spanning tree bounds. By projecting the SSP entirely onto a GTSP configuration graph, the non-compact formulation inherently bypasses the weak linear relaxations of classical models. Furthermore, it allows the Branch-and-Price framework to exploit the tight bounds of the GTSP polytope. As demonstrated by Fischetti et al. (1997), separating and adding Generalized Subtour Elimination Constraints (GSECs) can push the lower bound extremely close to the optimal integer solution at the root node.

The Pricing Bottleneck. The synthesis of JGP-derived columns, metaheuristic upper bounds, and GTSP-based lower bounds provides a robust Branch-and-Price framework capable of exact SSP resolution. However, a critical barrier remains. While the JGP's pure Set Covering oracle can be solved dynamically, the pricing oracle for the continuous SSP must simultaneously maximize dual variable collection while accounting for sequence-dependent transition costs. This renders the oracle strongly NP-hard. Solving this exact pricing problem at every column generation iteration dominates the computation time, necessitating the integration of Machine Learning to intelligently accelerate column generation.

4.4 Tool-Dependent Setup Times and Lexicographical Minimization

In the preceding sections, we established that the Job Grouping Problem (JGP) is structurally rigid and provides only an upper bound (α-approximation) for the uniform SSP. However, the operational reality of flexible manufacturing often involves *tool-dependent setup times* (the non-uniform SSP variant), where different tools require significantly different amounts of time or human effort to load into the magazine.

The Failure of KTNS: In the uniform SSP, sequence evaluation is solved in $O(mn)$ time via the greedy Keep Tool Needed Soonest (KTNS) policy. However, as proved by Crama and Oerlemans (1994), when setup times b_k are tool-dependent, the greedy optimal substructure of KTNS collapses. Instead, the tooling subproblem must be formulated as a 0-1 Linear Program to maximize the saved setup costs of retained tools. Because the constraint matrix exhibits the consecutive ones property, it forms an *interval matrix*. While interval matrices are totally unimodular and solvable in polynomial time via linear relaxation Crama and Oerlemans (1994), executing a full LP for *every* continuous sequence transition

during routing renders the continuous SSP computationally devastating.

Operational Reality and Lexicographical Minimization: To overcome this computational barrier, we look to practical manufacturing operations: when tool load times are highly variable, it is most efficient to halt the machine entirely and execute all necessary tool changes simultaneously in parallel.

Let c_{switch} be the average operational cost of an individual tool switch, and let C_{stop} be the fixed penalty of halting the machine to execute a batch of tool swaps. Because tools are swapped in parallel during a machine halt, the duration of the halt is bounded by the maximum individual load time ($\max_k b_k$) rather than the sum of load times ($\sum b_k$). This heavily inflates the value of C_{stop} relative to c_{switch}.

If $C_{stop} \gg c_{switch} \times b$, the objective function transitions into a **Lexicographical Minimization** framework Adjiashvili et al. (2015):

1. **Primary Objective:** Minimize the total number of full machine stops (the exact JGP Set Covering objective).
2. **Secondary Objective:** Minimize the individual tool switches within those defined boundaries (the SSP routing objective).

Recent literature proves that any optimal JGP stop plan can be mapped directly to a feasible, optimal tool switching plan for this primary objective Adjiashvili et al. (2015). Therefore, when C_{stop} dominates due to tool-dependent setup times, generating maximal configurations via the JGP is not merely a heuristic relaxation for the uniform SSP; it becomes the exact mathematical bound and optimal operational strategy for the non-uniform variant.

5 The Pricing Oracle and the Set Union Knapsack Problem

The previous sections established a profound link between two classical Combinatorial Optimization problems: the JGP provides the structurally optimal vertices (configurations), while the GTSP routing framework evaluates the continuous transitions between them. However, generating these JGP configurations dynamically within a non-compact Branch-and-Price (B&P) framework introduces a severe computational bottleneck in the form of the Pricing Oracle.

In the non-compact B&P framework, the Restricted Master Problem (RMP) selects configurations, but the **Pricing Subproblem** must iteratively generate new maximal configurations C (where $|C| = b$) with negative reduced costs. Given the dual variables $\pi_i \geq 0$

obtained from the RMP's job-covering constraints, the pricing problem seeks a configuration C that covers a subset of jobs $S \subseteq J$ to maximize the collected duals $\sum_{i \in S} \pi_i$, subject to the magazine capacity $|\bigcup_{i \in S} T_i| \leq b$.

Mathematical Equivalence to the SUKP: This exact mathematical structure is identical to the **Set Union Knapsack Problem (SUKP)** Goldschmidt et al. (1994). In the SUKP, items (jobs) have profits (π_i), and each item requires a set of elements (tools T_i). The weight of a subset of items is defined strictly by the cardinality of the union of their elements.

The SUKP is known to be strongly NP-hard. Exact dynamic programming algorithms for solving the SUKP exhibit an exponential worst-case complexity proportional to:

$$O\left(mnb \sum_{j=1}^{n} 2^{|h(j)|}\right) \tag{16}$$

where $h(j)$ denotes the set of elements adjacent to element j in the item-adjacency graph Goldschmidt et al. (1994).

Because the exact formulation of the SSP pricing oracle is fundamentally an SUKP, solving it to optimality at every iteration represents the primary computational barrier preventing the scalability of the exact B&P framework. This exponential complexity barrier explicitly necessitates the development of Machine Learning-based pricing heuristics (such as Neural Column Generation or Deep Reinforcement Learning) to approximate the oracle and accelerate the solver, which we explore in the subsequent sections.

6 Exact Routing for Tractable Instances

As established by the complexity framework, generating the complete configuration graph is computationally viable only for smaller, tightly constrained instances where the combination of m and b does not cause the state space $\mathcal{O}(|J|\binom{m}{b})$ to exceed memory limits. For these tractable instances, we can leverage highly optimized, state-of-the-art routing solvers by applying a secondary polynomial transformation.

Once the overlapping SSP clusters are expanded into mutually exclusive GTSP clusters via the I-N transformation, the problem can be reduced to a standard Asymmetric Traveling Salesman Problem (ATSP). This is achieved through the Noon-Bean transformation (Noon and Bean, 1993), which systematically modifies the intra-cluster and inter-cluster edge weights to logically compel any exact routing algorithm to visit exactly one node per cluster.

By collapsing the generalized problem into an ATSP, the formulation can be passed directly to advanced routing heuristics and exact solvers, such as LKH-3 (Helsgaun, 2015). While this approach yields optimal or near-optimal solutions rapidly for small instances, its reliance on the full, explicit enumeration of the configuration graph renders it intractable for massive industrial applications, necessitating dynamic node generation.

7 Dynamic Node Generation and the True Pricing Bottleneck

For large-scale instances, the exponential size of the vertex set $V(K_p)$ makes it impossible to define the complete problem space upfront. Instead, we transition from the non-compact formulation to a Branch-and-Price paradigm. The problem is decomposed into a Restricted Master Problem (RMP), which operates over a small, generated subset of valid configuration sequences (columns), and a pricing oracle, which dynamically generates new sequences on-the-fly.

The RMP is formulated as a Set Covering problem, selecting a combination of generated sequences to satisfy all job requirements at a minimal transition cost. Let $\pi_j \geq 0$ be the dual variables associated with the covering constraints of the RMP for each job $j \in J$.

The pricing oracle's objective is to find a new sequence of configurations (a column) whose inclusion in the RMP could improve the objective function. Mathematically, it must identify a sequence with a negative reduced cost $\bar{c}$. For a newly generated continuous sequence of configurations C, the reduced cost is calculated as:

$$\bar{c} = \sum_{(u,v) \in E(C)} d(u,v) - \sum_{j \in J(C)} \pi_j \tag{17}$$

where $E(C)$ represents the edges of the generated configuration sequence, $d(u,v)$ is the tool switching transition cost, and $J(C)$ is the subset of jobs covered by the sequence.

Proposition 7.1 (NP-Hardness of the SSP Pricing Oracle)**.** *The pricing problem for the non-compact SSP formulation is strongly NP-hard.*

Proof. The pricing oracle requires identifying a valid sequence of configurations $C \in \mathcal{C}$ that minimizes the reduced cost $\bar{c}$ defined in Equation (17). This requires simultaneously determining an optimal subset of jobs $J(C)$ to cover (to maximize the collected dual variables π_j) while finding the shortest valid tool-switching sequence to process them (to minimize the transition costs $d(u,v)$). This structure perfectly mirrors the Prize-Collecting Traveling

Salesman Problem (PCTSP), where dual variables act as collected prizes and switching costs act as routing penalties. Because the PCTSP is inherently strongly NP-hard, identifying the column with the minimum reduced cost for the SSP is also strongly NP-hard. □

It is crucial to distinguish this pricing oracle from the Job Grouping Problem (JGP). The JGP acts as a pure Set Union Knapsack Problem because it assumes the machine is completely halted for simultaneous batch swaps. In contrast, the sequential SSP pricing oracle must account for continuous transition costs $d(u, v)$ along the edges. Because solving this NP-hard oracle exactly via Mixed-Integer Linear Programming (MILP) at every column generation iteration dominates the total computation time, accelerating the oracle is the primary focus of our future experimental roadmap.

8 Machine Learning Interventions

To overcome the exact pricing bottleneck, we propose integrating Machine Learning (ML) techniques at critical stages of the Branch-and-Price algorithm. By substituting expensive exact computations with rapid, learned approximations, we can accelerate convergence while maintaining global optimality guarantees.

8.1 Pricing Graph Pruning via GNNs

At each iteration, the exact MILP pricing oracle searches for paths across the configuration graph. Because the majority of configurations will never yield a negative reduced cost, searching the entire space is highly inefficient. We propose training a Graph Neural Network (GNN) to predict the likelihood of a configuration belonging to an optimal column based on the current dual variables π_j. By filtering out low-probability nodes, the GNN drastically prunes the pricing graph, reducing the search space for the exact MILP solver in subsequent iterations.

8.2 Reinforcement Learning for Column Selection

A classic approach in column generation is to add multiple columns at each iteration rather than just the single most negative one. However, highly correlated columns lead to degeneracy in the RMP. The RMP naturally forms a bipartite graph between jobs and configurations. We can train a Deep Reinforcement Learning (RL) agent to evaluate this bipartite state and select a diverse batch of mutually disjoint configurations. Injecting varied, high-quality columns into the RMP at each iteration mitigates degeneracy and minimizes the total

number of column generation iterations required. Furthermore, adapting the Ryan-Foster branching rule—originally designed for Set Partitioning (equality constraints)—to handle the SSP's Set Covering (inequality constraints) structure provides a fertile ground for RL branching agents to discover optimal branching variables.

8.3 Feasibility Checking via Sequence Models

A major computational burden within the pricing oracle is verifying whether a generated sequence of jobs respects the tool magazine capacity across all transitions. **?** implement a Neural Column Generation (NCG) pipeline where an Attention-based sequence model predicts the feasibility of columns prior to invoking an expensive exact feasibility checker. By rejecting structurally infeasible sequences early using the sequence model, we bypass unnecessary computational overhead.

8.4 Regression for Bound Prediction

Finally, the exact MILP solver used in the pricing oracle can be significantly accelerated if provided with a tight upper bound for its objective function. Following the framework proposed by Václavík et al. (2018), we will deploy a Regression model to predict the expected minimum reduced cost at each iteration based on historical dual variable values. Passing this predicted bound to the MILP solver enables aggressive pruning of its own search tree, drastically reducing the time spent proving optimality.

9 Experimental Roadmap

The theoretical properties established in this report pave the way for a highly structured experimental plan. We outline a phased integration of Machine Learning methods to progressively accelerate the exact algorithm, summarized in Table 1.

Phase	Description	RMP Init.	Pricing Graph	Oracle Generator	Column Select.	Exact Fallback
0	Baseline	Random	Full Graph	Exact MILP	Greedy (Std)	Exact MILP
1	OR Enhancements	Multi-Start Heuristic	Full Graph	Exact MILP	Greedy (Std)	Exact MILP
2	Exact Acceleration	Best from Phase 1	GNN-Pruned	Exact MILP	Greedy	Regress. Bound
3	Neural Generators	Best from Phase 1	GNN-Pruned	MLPH / GNN	Diverse-M (RL)	Regress. Bound
4	SOTA Synthesis	Best from Phase 1	GNN-Pruned	Attn-Sequence	Deep RL	Regress. Bound

Table 1: Experimental Roadmap for ML-Accelerated Exact Branch-and-Price.

Phase 0 establishes the computational baseline using a standard Branch-and-Price framework. Phase 1 introduces pure Operations Research (OR) enhancements, utilizing advanced multi-start heuristics to seed the RMP. Phase 2 introduces exact ML accelerators, deploying GNNs to prune the pricing graph and Regression models to predict bounds for the MILP solver. Phase 3 replaces the exact oracle entirely with Neural Generators (MLPH/GNN) for the bulk of the iterations. Finally, Phase 4 represents the state-of-the-art synthesis: using Sequence Models to rapidly generate columns, Deep RL to select a disjoint batch of them, and Regression Bound Prediction to accelerate the final, mandatory exact proof of optimality.

10 Conclusion

This report formalized the mathematical properties of a non-compact, configuration-based formulation for the Uniform Tool Switching Problem (SSP). By establishing the structural subsumption of the Generalized Traveling Salesman Problem (GTSP) within the SSP under the strict Disjointness Condition, we successfully proved that the metric barrier prevents a generalized polynomial reduction. This complexity framework mathematically justifies a bifurcated algorithmic approach: while tractable instances can be reduced to the Asymmetric TSP and solved via highly optimized routing heuristics, industrial-scale instances necessitate dynamic node generation. Building upon this rigorous foundation, we proposed a progressive Experimental Roadmap designed to leverage Machine Learning—specifically Graph Neural Networks, Deep Reinforcement Learning, and Regression—to bypass the NP-hard pricing bottleneck within an exact Branch-and-Price paradigm.

References

David Adjiashvili, Sandro Bosio, and Kevin Zemmer. Minimizing the number of switch instances on a flexible machine in polynomial time. *Operations Research Letters*, 43(3): 317–322, 2015.

A. C. Beezão, J.-F. Cordeau, G. Laporte, and H. H. Yanasse. Scheduling identical parallel machines with tooling constraints. *European Journal of Operational Research*, 257(3): 834–844, 2017.

László A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.

Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.

Rafael Colares and Annegret Wagler. Exact formulations for the job sequencing and tool switching problem. *Working Paper*, 2026.

Yves Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1):58–80, 1994.

Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3): 378–394, 1997.

M. Furrer and T. Mütze. An algorithmic framework for tool switching problems with multiple objectives. *European Journal of Operational Research*, 259(3):1003–1016, 2017.

O. Goldschmidt, D. Nehme, and G. Yu. Note on the set-union knapsack problem. *Naval Research Logistics (NRL)*, 41(6):833–842, 1994.

Keld Helsgaun. Solving the equality generalized traveling salesman problem using the lin–kernighan–helsgaun algorithm. *Mathematical Programming Computation*, 7(3):269–287, 2015.

Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.

Yao-Nan Lien, Eva Ma, and Benjamin W. S. Wah. Transformation of the generalized traveling-salesman problem into the standard traveling-salesman problem. *Information Sciences*, 74(1-2):177–189, 1993.

Jordana Mecler, Anand Subramanian, and Thibaut Vidal. A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *Computers & Operations Research*, 127:105153, 2021.

Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.

Caroline Privault and Gerd Finke. Modelling a tool switching problem on a single nc-machine. *Journal of Intelligent Manufacturing*, 6(2):87–94, 1995.

Caroline Privault and Gerd Finke. K-server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research*, 96(1/4):255–269, 2000.

Yunzhuang Shen, Yuan Sun, Xiaodong Li, Andrew Eberhard, and Andreas Ernst. Enhancing column generation by a machine-learning-based pricing heuristic for graph coloring. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9926–9934, 2022.

Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5): 767–777, 1988.

Robin Václavík, Antonín Novák, Přemysl Šůcha, and Zdeněk Hanzálek. Accelerating the branch-and-price algorithm using machine learning. *European Journal of Operational Research*, 271(3):1055–1069, 2018.